```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure
```

```
import cv2

print("OpenCV version:", cv2.__version__)
print("OpenCV is working!")
```

```
OpenCV version: 4.10.0
OpenCV is working!
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# Create dataset folder
os.makedirs("dataset", exist_ok=True)

# Create a sample dataset image
img = np.ones((500, 500, 3), dtype=np.uint8) * 255

# Add different shapes and text
cv2.rectangle(img, (50, 50), (450, 450), (0, 0, 0), 4)
cv2.circle(img, (250, 250), 120, (0, 0, 0), 4)

cv2.line(img, (50, 50), (450, 450), (0, 0, 0), 4)
cv2.line(img, (450, 50), (50, 450), (0, 0, 0), 4)

cv2.putText(
    img,
    "SIFT",
    (150, 480),
    cv2.FONT_HERSHEY_SIMPLEX,
    1.2,
    (0, 0, 0),
    3
)

# Save image
cv2.imwrite("dataset/image1.jpg", img)

print("Dataset created!")
print("File: dataset/image1.jpg")

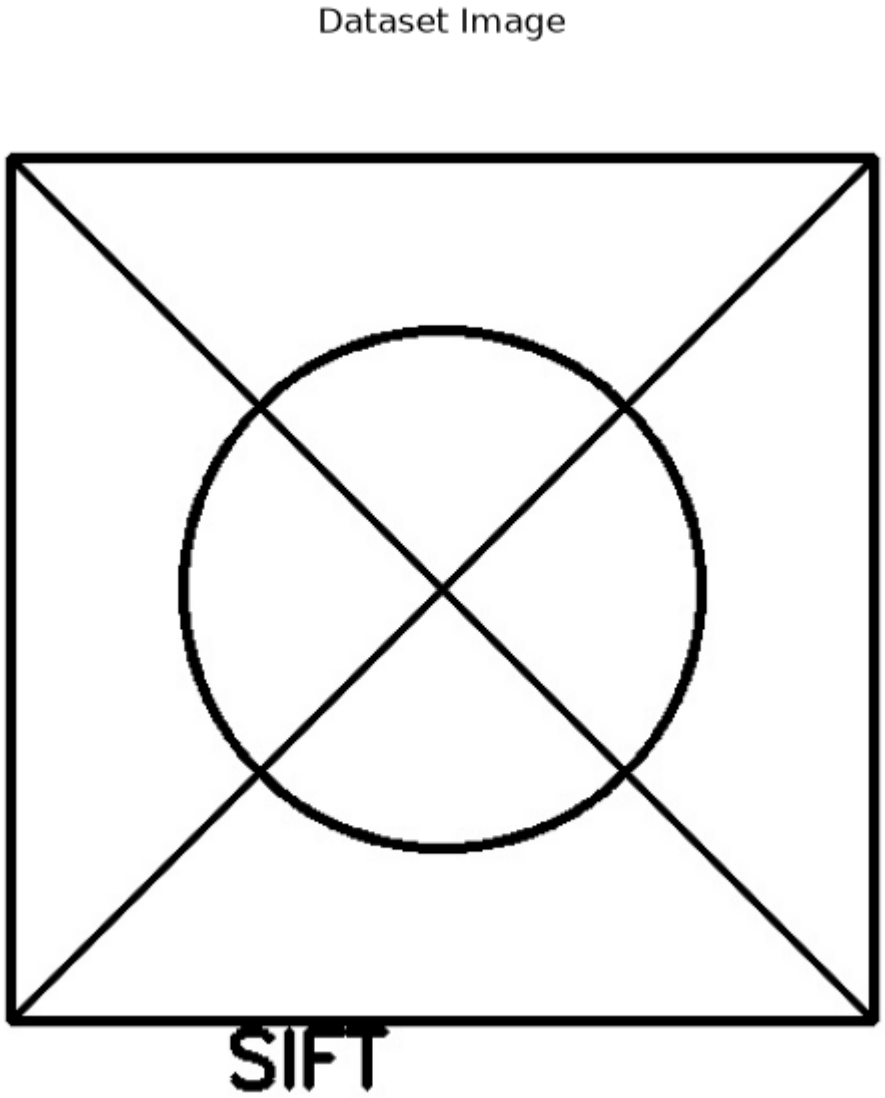

# Load dataset
image = cv2.imread("dataset/image1.jpg")

# Display dataset
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(7, 7))
plt.imshow(image_rgb)
plt.title("Dataset Image")
```

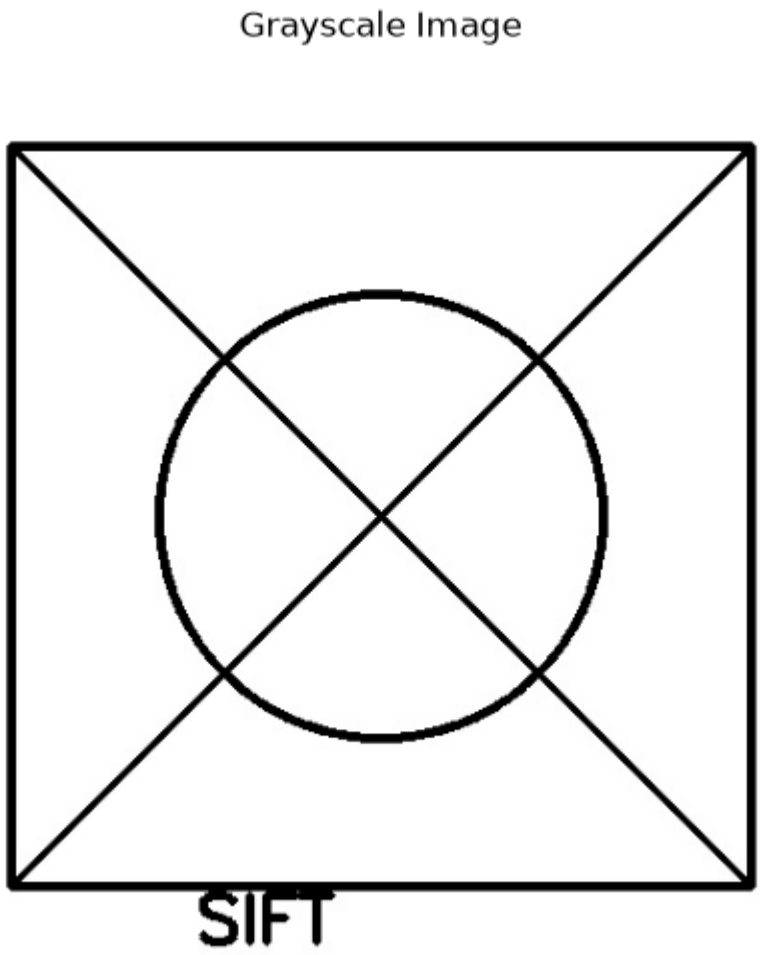
```
plt.axis("off")
plt.show()
```

```
Dataset created!
File: dataset/image1.jpg
```




```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8, 6))
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()
```




```
# Create SIFT detector
sift = cv2.SIFT_create()

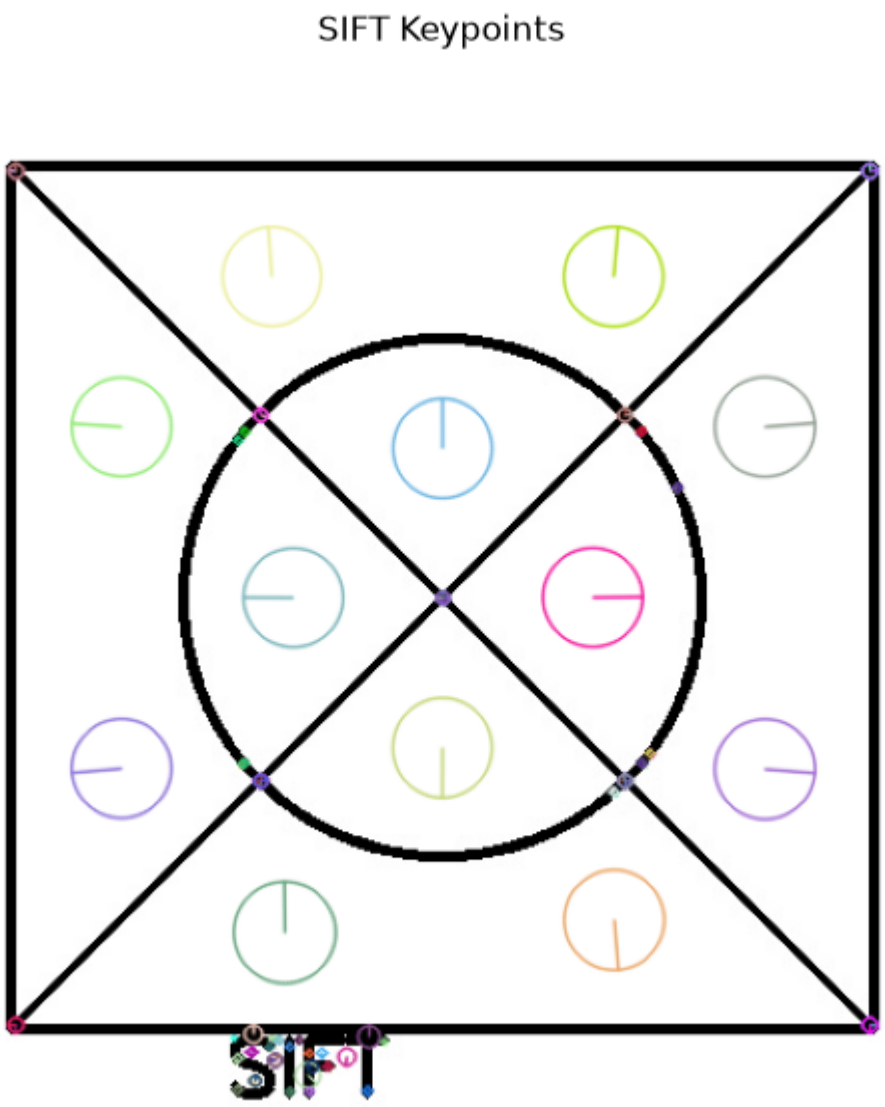
# Detect keypoints and calculate descriptors
keypoints, descriptors = sift.detectAndCompute(gray, None)

print("Number of keypoints detected:", len(keypoints))
print("Descriptor shape:", descriptors.shape)
```

```
Number of keypoints detected: 83
Descriptor shape: (83, 128)
```

```
sift_image = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(10, 7))
plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()
```




```
# Resize image for HOG
hog_image = cv2.resize(gray, (128, 128))

# Extract HOG features
features, hog_visualization = hog(
    hog_image,
    orientations=9,
    pixels_per_cell=(8, 8),
    cells_per_block=(2, 2),
    visualize=True,
    block_norm='L2-Hys'
)

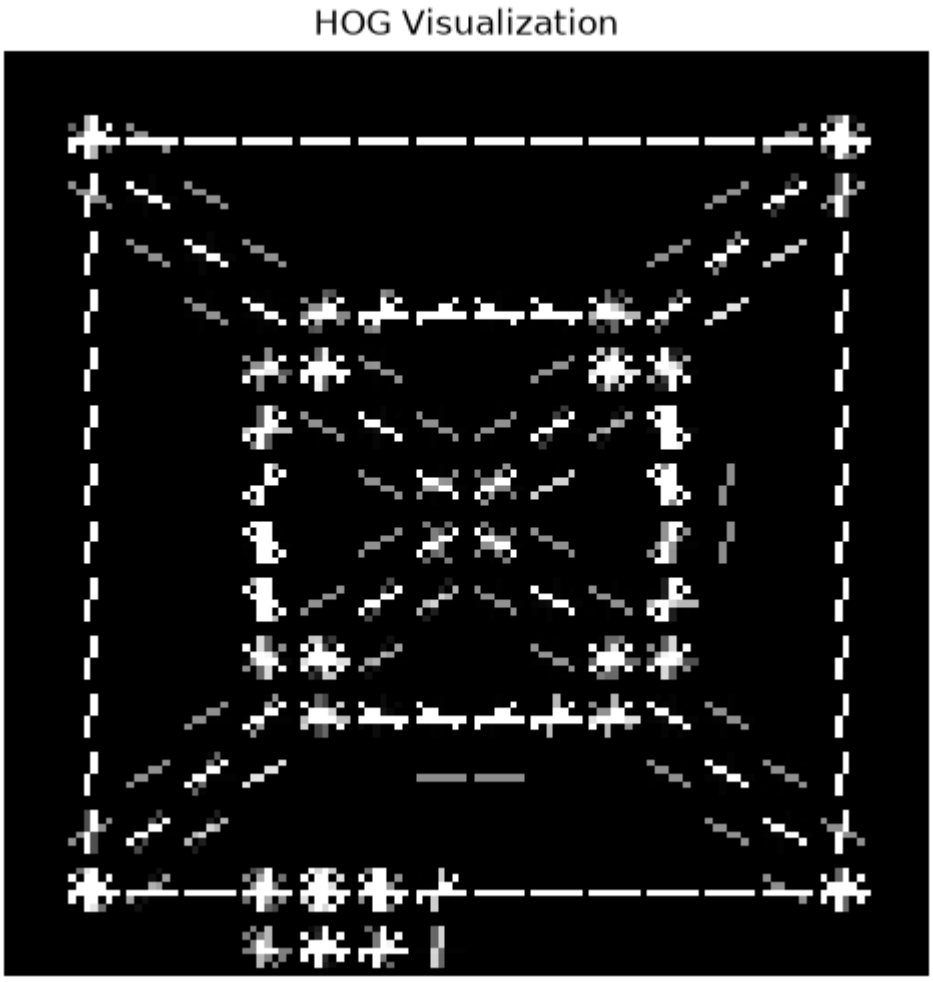
print("HOG feature vector length:", len(features))
```

```
HOG feature vector length: 8100
```

```
# Improve HOG visualization
hog_visualization = exposure.rescale_intensity(
    hog_visualization,
    in_range=(0, 10)
)

plt.figure(figsize=(8, 6))
plt.imshow(hog_visualization, cmap="gray")
```

```
plt.title("HOG Visualization")
plt.axis("off")
plt.show()
```




```
print("----- SIFT -----")
print("Number of keypoints:", len(keypoints))
print("Descriptor dimensions:", descriptors.shape)

print("\n----- HOG -----")
print("HOG feature vector length:", len(features))
```

```
----- SIFT -----
Number of keypoints: 83
Descriptor dimensions: (83, 128)

----- HOG -----
HOG feature vector length: 8100
```

```
import cv2
import os

image2 = cv2.imread("image2.jpg")

if image2 is None:
    print("Image nahi mili. Check karo ki image2.jpg same folder me hai.")
else:
    gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

    # Detect SIFT features
    keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)
```

```
    print("Image loaded successfully!")
    print("Keypoints:", len(keypoints2))
```

```
Image nahi mili. Check karo ki image2.jpg same folder me hai.
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# image1 load karo
image1 = cv2.imread("dataset/image1.jpg")

if image1 is None:
    print("image1.jpg nahi mili")
else:
    print("Image 1 loaded successfully!")

# Image 1 ko thoda rotate karo
height, width = image1.shape[:2]
center = (width // 2, height // 2)

matrix = cv2.getRotationMatrix2D(center, 10, 1.0)

image2 = cv2.warpAffine(image1, matrix, (width, height))

# Image 2 save karo
cv2.imwrite("dataset/image2.jpg", image2)

print("Image 2 created successfully!")
```

```
Image 1 loaded successfully!
Image 2 created successfully!
```

```
# Load both images
image1 = cv2.imread("dataset/image1.jpg")
image2 = cv2.imread("dataset/image2.jpg")

# Convert to grayscale
gray1 = cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(image2, cv2.COLOR_BGR2GRAY)

# Create SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and descriptors
keypoints1, descriptors1 = sift.detectAndCompute(gray1, None)
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

print("Image 1 keypoints:", len(keypoints1))
print("Image 2 keypoints:", len(keypoints2))
```

```
Image 1 keypoints: 83
Image 2 keypoints: 186
```

```
# Brute Force Matcher
bf = cv2.BFMatcher()

matches = bf.knnMatch(descriptors1, descriptors2, k=2)

# Lowe's ratio test
```

```
good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Total matches:", len(matches))
print("Good matches:", len(good_matches))
```

```
Total matches: 83
Good matches: 49
```

```
matched_image = cv2.drawMatches(
    image1,
    keypoints1,
    image2,
    keypoints2,
    good_matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(15, 7))
plt.imshow(cv2.cvtColor(matched_image, cv2.COLOR_BGR2RGB))
plt.title("SIFT Feature Matching")
plt.axis("off")
plt.show()
```

